# Diving into Lucene

Andy Tang

05/10/2008

# Table of contents

**1. Introduction** ..... **1**
**2. The installation** ..... **2**
2.1. Preparation ..... 2
2.2. Java package installation ..... 2
2.3. Apache Tomcat installation ..... 3
2.4. Lucene package installation ..... 4
2.5. JUNIT package installation ..... 4
2.6. Apache Ant package installation ..... 5
**3. The Lucene structure** ..... **6**
**4. The functionality** ..... **7**
4.1. Indexing ..... 7
4.1.1. Conversion to text ..... 7
4.1.2. Index Analysis ..... 8
4.1.3. Index storing ..... 9
4.1.4. Index demonstration ..... 10
4.2. Query and basic search ..... 11
4.2.1. Research ..... 11
4.2.2. Query Demonstration ..... 13
4.3. Advanced search ..... 14
4.3.1. Multiple indexes ..... 14
4.3.2. Term vectors leverage ..... 16
4.3.3. Customization ..... 17
4.3.4. Scoring function ..... 18
4.4. Advanced function demonstration on Lucene ..... 20
4.4.1. Analyzer demonstration ..... 20
4.4.2. Vector leverage demonstration ..... 21
4.4.3. Search results Sorting demonstration ..... 22
4.4.4. Synonym analyzer demonstration ..... 23
**5. Sandbox in Lucene** ..... **24**
**6. Different platforms of Lucene** ..... **28**
**7. Conclusion** ..... **28**

# List of figures

Figure 2.1 ····3
Figure 2.2 ····3
Figure 2.3 ····4
Figure 2.4 ····5
Figure 2.5 ····5
Figure 3.6 ····7
Figure 4.7 ····8
Figure 4.8 ····10
Figure 4.9 ····11
Figure 4.10 ····11
Figure 4.11 ····13
Figure 4.12 ····14
Figure 4.13 ····15
Figure 4.14 ····16
Figure 4.15 ····17
Figure 4.16 ····21
Figure 4.17 ····22
Figure 4.18 ····23
Figure 4.19 ····24
Figure 5.20 ····25
Figure 5.21 ····26
Figure 5.22 ····27
Figure 5.23 ····27

## 1. Introduction

Facing fast expansion of internet and increased information requirements in modern society, it is very important to research on how to match user's queries with the information distributed in every corner. Gradually, information retrieval becomes the focus and highlight of this research area, and many professionals put their whole-life energy and strength into it. Many of them make great contribution, and push our society forward by increasing the information usability. Among these projects and achievements, Lucene is the most shining one.

Lucene is a high performance, fast developed, highly-organized, scalar information retrieval library, which consists of many useful APIs, used in building strong information retrieval system (Gospodnetic &Hatcher, 2005). By simile, lucene is like a pile of gold bricks. Information engineers can use them to build a palace, temple, or eloquent castle, depending on concrete aims or targets. By using different combination method, they can produce different magic products. Hence, Lucene is a highly organized toolkits box with great application.

Lucene provides abundant functions through API forms. Nearly all functions, used in building a complete IR system, such as index building, query parsing, high-effect searching, and many special techniques, are delicately forged and provided. Many functions are analyzed deeply in this paper. However, for the reason of so large a system, it is impossible and unnecessary to pick up and study every details and aspects of Lucene. Instead, it is a good way to choose some main aspects to depict, so as to form a solid frame in understanding Lucene.

Lucene was born as a self-serving project in late 1997. Lucene's creator, Doug Cutting, has great theoretical and practical experience in the field of IR, publishing many research papers in this area (Gospodnetic &Hatcher, 2005). In 2000, for some individual reason, Doug decides to make Lucene an open source project. As a result, in 2001, Apache project draw in Lucene under its flag. From that time, many high-quality contribution codes are infused into Lucene. From 2004, Lucene begin to hold a group of mature developers with deep understanding on Lucene core. Nowadays, Lucene has evolved into 2.3.2 version on Java platform.

## 2. The installation

The most wonderful starting point to understand an information retrieval tools is to install it. The installation of Lucene 2.3.2, involving java software package setup, is a little complicated, especially on Microsoft windows platform.

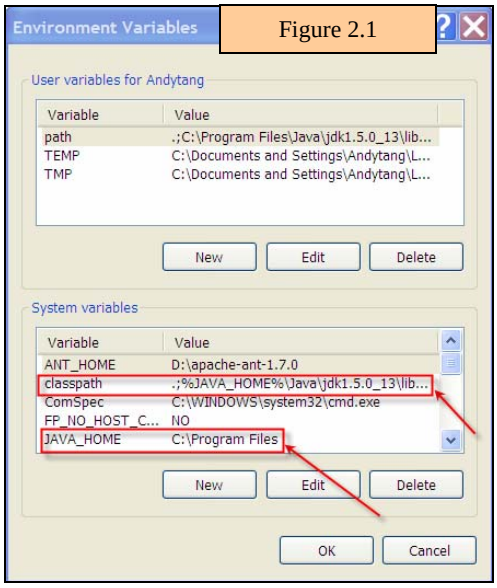
### 2.1. Preparation

Before installation, some adequate preparation must be made, such as holding Java package, Apache tomcat package, Lucene-java-2.3.1 package, JUNIT package, and Ant package. Some basic knowledge from Java are absolutely required, otherwise there will a great difficulties in understanding and comprehension.

### 2.2. Java package installation

The first step, JAVA software package must be installed, because it is the indispensable platform to run the Lucerne-java. The version of JAVA chosen is jdk-1.5.0.13 for x86 windows. After installation complete, it is important to add the location of Java into operating system's path variable (Figure 2.1); otherwise it can not work well.

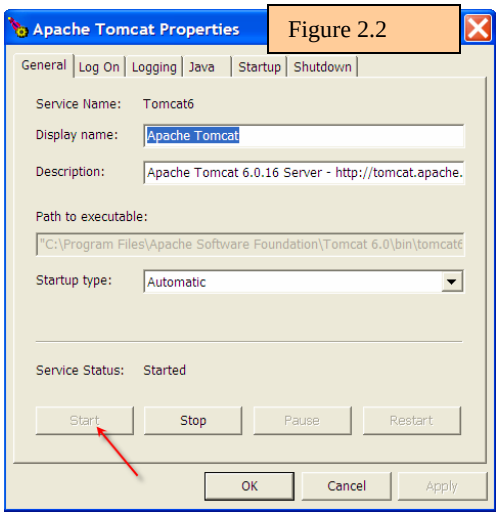
Figure 2.1




## 2.3. Apache Tomcat installation

The second step, Apache tomcat software package must be installed. Tomcat is a web container, or application server, which mainly implements the Java Servlet and the JavaServer Pages (JSP) (Apache researcher, 2005). It definitely provides a convenient environment, and platform, for Java code running. The version of Tomcat, we choose, is 6.0.16. Apache Tomcat itself can setup the path automatically, convenient for users (Figure 2.2). The Apache Tomcat downloading website is http://tomcat.apache.org/download-60.cgi.

Figure 2.2



## 2.4. Lucene package installation

The third step, Lucene software package must be installed. The downloading address is http://www.apache.org/dyn/closer.cgi/lucene/java/. Although there are many versions can be chosen, the most perfect version definitely should be Lucene-JAVA-2.3.2. After installation, the CLASSPATH below must be added into system variables. Simultaneously, it is absolutely required to copy the files lucene-core-2.3.1.jar and lucene-demos-2.3.1.jar from Lucene directory into JAVA lib directory. After this step, I build a directory, named "d:\lucenetest\index", to store the original files required to be indexed.

```
CLASSPATH： .;%JAVA_HOME%\lib\tools.jar;%JAVA_HOME%\lib\l
ucene-core-2.3.1.jar;%JAVA  HOME%\lib\lucene-demos-2.3.1.jar
```

Under the D:\lucenetest\index directory, there are 6 text files and one sub directory named HTML which contains another 9 mixed files. In total, there are 15 files in test. (Figure 2.3).

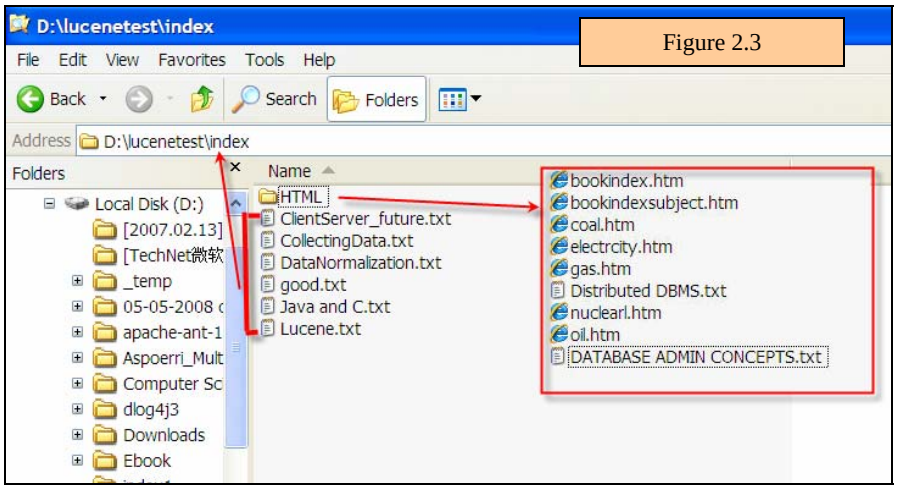



Figure 2.3

## 2.5. JUNIT package installation

The fourth step, JUNIT (Java unit-testing framework) software package must be installed in advance, on the one hand, to help Ant software work smoothly, on the other hand, to ensure Lucene-Java unit running perfectly. One point should be paid attention that the file of junit.jar should be placed under ANT_HOME/lib directory, otherwise, Apache Ant software package can not run well on Lucene code. The newest version of JUNIT chosen is 4.4. The downloading address is http://junit.org (Figure 2.4).

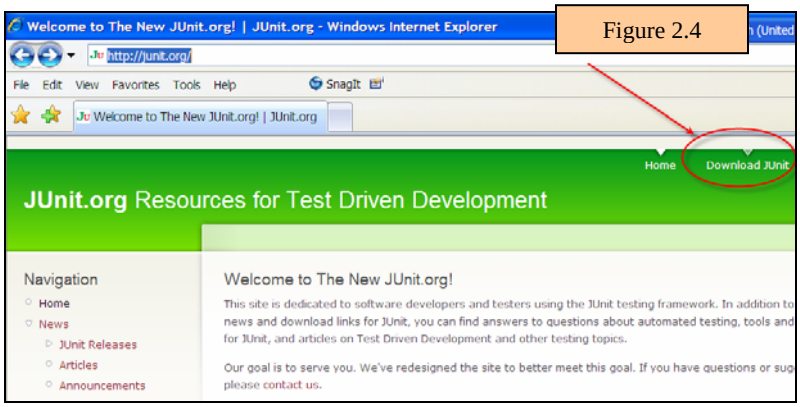



Figure 2.4

## 2.6. Apache Ant package installation

The last step, Apache Ant software package must be installed. Apache Ant is a convenient software platform in automating java software build process (Apache Ant researcher, 2005). Moreover, Apache Ant uses XML to describe the build process, a wonderful way in building Java projects. The Apache Ant version chosen is 1.70. The downloading website is http://ant.apache.org/. After installation, the Ant path must be added into system variable according to the Figure 2.5.

Figure 2.5
```
Set   ANT_HOME=d:\ apache-ant-1.7.0
Set   PATH=%PATH%; %ANT_HOME%\bin
```

Whether the IR system can be installed easily and conveniently, is very important, especially for users. A friendly software package can save lots of time for users, and

improve the work efficiency. For the reason of complicacy of Java system, and involving the web page server configuration, every step of installation is not easy, not only for common users, but also for specialists and professionals. Furthermore, the platforms of apache software series updates very fast, making users reinstall it inevitably. Hence, from the vision of usability, Lucene installation, together with other Java series package, are too much complicated to handle easily.

**3. The Lucene structure**

The Lucene software platform provides lots of tool kits and API to complete the building functions (Figure 3.6). Actually, there exists a seamless connection among its components. It is helpful to know about the whole Lucene if understanding the relationship between its components. The illustration below demonstrates the connections or relationships among different Lucene components.

The first, Lucene is a support platform, providing functions and supports for software on application layers. The second, Lucene communicates with application layers through two important channels, which are separately index and search module. The third, Lucene, in a role of support, consists of many APIs, which can be integrated or disintegrated like toy bricks. The fourth, Lucene, besides providing bottom layer API support, have a tendency in extending its functions toward top layers which originally belong to application ones. In total, Lucene are in the process of development, with brilliant future.

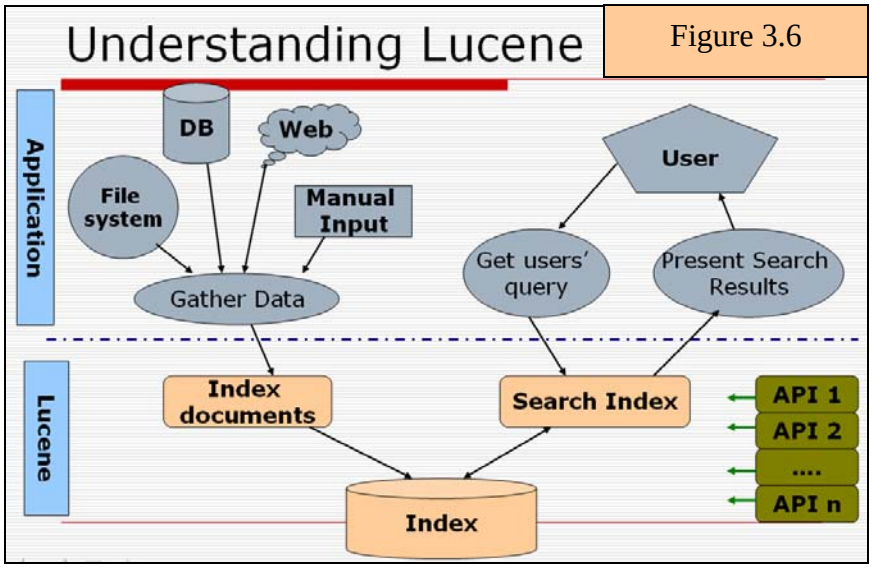



Figure 3.6

## 4. The functionality

### 4.1. Indexing

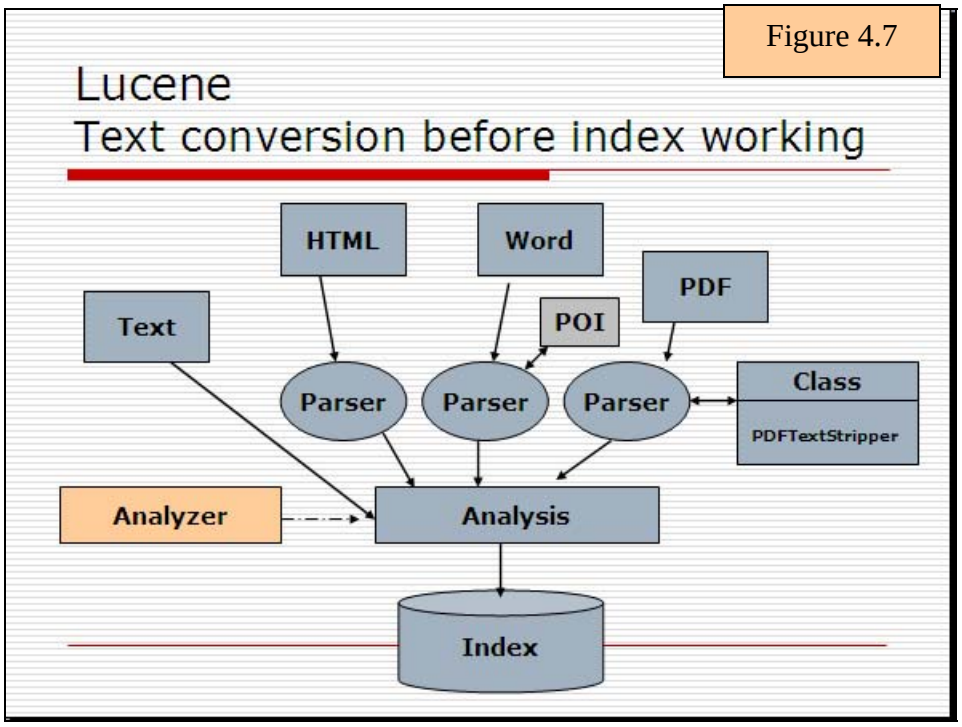
Indexing, in Lucene, is so important that it becomes the bridge between search functions and original information resources. Actually index is the base and foundation of Lucene platform. It encompasses three important functions which separately are conversion one, parsing one, and index storing one.

#### 4.1.1. Conversion to text

Conversion to text is the first important function of Lucene index. The basic file format Lucene can directly handle is plain text file format. However, in real world, file format are abundant and affluent, including rich-text format, such as pdf, html, doc file format, etc. How to handle these file format is a big problem facing to Lucene. Fortunately, Lucene has a set of delicate methods to handle them, such as adopting the framework. For Microsoft word format, Lucene converts them to text by using POI (Apache POI researcher, 2005). For Adobe PDF format, it converts them to text by using

class named “PDFTextStripper”. The illustration below (Figure 4.7) clearly clarifies the conversion process from rich-text file format to plain-text file format.

Figure 4.7




### 4.1.2. Index Analysis

Analysis is the second important functions of Lucene index, mainly undertaken by analyzer. Analysis functions mainly include dealing with tokens, evaluating and erasing stop words, handling synonyms, do stemming, and so on. After parsing phrase, the plain-text is subsequently handled by analyzer.

Tokenization is responsible for dealing with tokens in documents, with position increment and offsets (Gospodnetic &Hatcher, 2005). After tokens are analyzed, when they are stored into index, tokens are changed into terms.

The stop words removal functions are handled by stopfilter class (Class is a procedural block used to group related attributes and methods in software). The first step, all tokens are converted to low case. The second step, all little words are removed

according to stop words list. However, there is another function which allows users to pass their own stop word list to substitute the default ones (Gospodnetic &Hatcher, 2005), providing great flexibility. We will demonstrate these functions on later parts.

Lucene has strong functions to deal with synonyms or alias. During process of tokenization and analysis, analyzer can use the “SynonymEngine” (Gospodnetic &Hatcher, 2005) produce the synonym, then inject or push them into the same position (virtually) as the original word. Simultaneously, it will mark or tag every synonym’s place to make procedure can chain or connect them with original ones.

The stemming analysis is another strong function of Lucene. By using Porter stemming algorithm, which is invented by Martin Porter, various forms of terms with connection shrinks into a common root. This function abbreviates and simplifies the word handling process, but decrease the precision, because computers merely have a shallow understanding for semantic meaning and context (van Rijsbergen, 1979).

**4.1.3. Index storing**

Before entering into the topic of index storing, some basic understanding for index structure should have. Within the structure of Lucene index, field plays an important role. There are four kinds of fields in Lucene, which separately are Keyword, UnIndexed, Unstored, and Text. All fields include a name and corresponding value, which demonstrate a simple and succinct index structure easily to understand and to use.

After the phase of analyzing, all indexes are stored into physical disk. However, the data structure, Lucene store the index, is called inverted index, which means using tokens as lookup keys, instead of treating documents as the central entities (Gospodnetic

&Hatcher, 2005). Moreover, Lucene provide the function of sorting search results, but it requires adding fields in advance to ensure the sorting functions.

Lucene provide concurrency mechanism to ensure the efficiency of operation. Simultaneously, it also provides thread-safety and locking control to ensure the data integrity and decrease the operation collision on index.

On another side, to ensure the efficiency of document index, and to avoid the performance descending coming from frequent I/O, Lucene invents a method to keep a fixed number of documents into computer memory, and write them to disk together. By adopting this method, it not only improves the index making speed, but also improves the index searching speed.

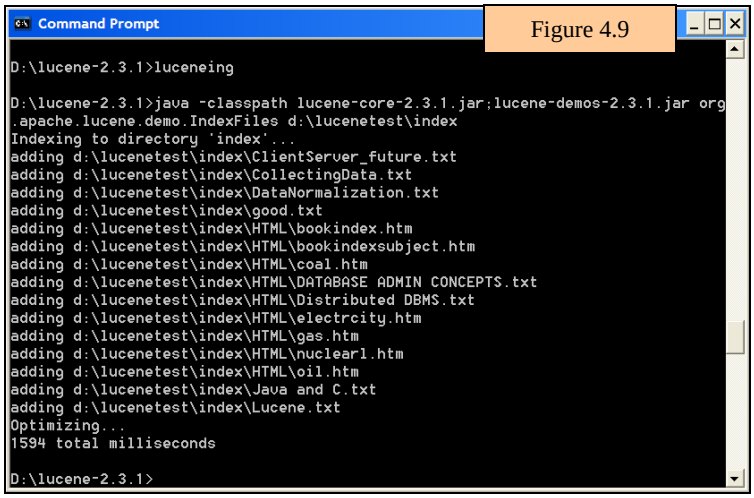
**4.1.4. Index demonstration**

A demonstration was made to present the index process and storing process. The first, we must go into the directory of Lucene. The second, we open the dos windows and input the command in illustration below. For convenience, we put the command line into a batch file, named “luceneing.bat”, and then we execute it (Figure 4.8).

Figure 4.8
Java -classpath lucene-core-2.3.1.jar; lucene-demos-2.3.1.jar
org.apache.lucene.demo.IndexFiles d:\lucenetest\index

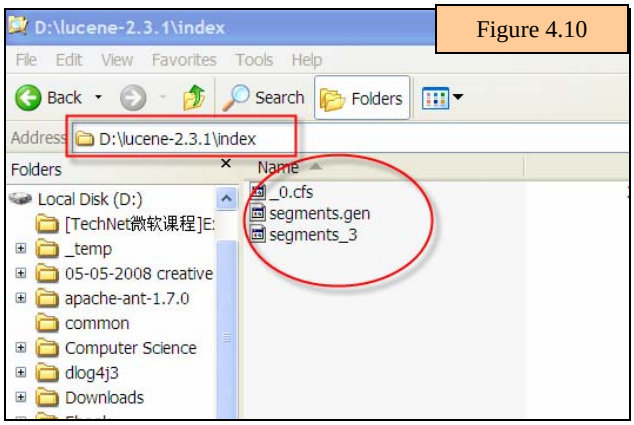
The results will display as below. We can find that the original files in the directory of “D:\lucenetest\index” are totally handled immediately, including 8 text format files and 7 html format files (Figure 4.9).

Figure 4.9




Lucene-java automatically analyzes any files and subdirectories in "D:\lucenetest\index", and put generated index into the directory "D:\lucene-2.3.1\index". We can see the file names of generated index under this directory, in the illustration below (Figure 4.10).

Figure 4.10




## 4.2. Query and basic search

### 4.2.1. Research

In order to satisfy the users' information needs, it is very important to analyze the users query and parse them, refine them. Lucene is a very practical system, not put its focus on how to teach or educate users to use complicated search methods. Instead, it provides many functions and API as a good preparation for building an effective

information retrieval and feedback system. After obtaining the query from users, query parser begins to go upon the stage.

The query management ability in Lucene is very strong. By combining query parser, Boolean query, range query, and term query functions, Lucene can effectively satisfy the information requirements coming from users. Moreover, Lucene provide a high technique to strengthen, refine, and clarify the human-centered query, by combining parsed query with an API-centered query (Gospodnetic &Hatcher, 2005).

Query parser is an analyzer or parser which takes effect on human centered query. Query parser can handle many kinds of queries, such as Boolean query, field selection, range query, phrase query, wildcard and prefix query, and fuzzy query. Term query parser is to check whether a single word in a query is equal to a term in documents. This function is nearly no different with other search techniques. However, other kinds of query parsers do have the strong ability and seldom adopted by other websites. The first one of them is range query parser which restricts the query factors into a fixed range, not only include the time range, but also include the text range. Boolean choice, combining field selection, gives a strong support to range query. The second one of them is phrase query parser which gives a detailed analysis for the phrase enclosed in double quotes. The third one of them is wildcard and prefix query parser, realized by analyzing asterisk sign and question mark in queries. The fourth one of them is fuzzy query parser, analyzing the trailing sign (~) which be put on the preceding position of term.

Among the query parsers, the most important one are boost query parser. Boost query is a useful tool to normalize the scores, mainly employed to determine which document in search results should be put on top position. Another important query parser

is span query, which includes many subroutines, such as, span-term query, spanning at the beginning of a field, spanning near one another, excluding span-overlap from matches, etc.

However, until now, the query parser class can not directly support the span functions. It must use the java procedure to connect different APIs to support and realize it. Maybe in future, it will support these functions directly through query parser.

**4.2.2. Query Demonstration**

A demonstration was made to represent the query process. The first, we must go into the directory of Lucene. The second, we open the Dos windows and input the command in illustration below (Figure 4.11). For convenience, we put the command line into a batch file, named “luceneing2.bat”, and then we execute it.

Figure 4.11
Java -classpath lucene-core-2.3.1.jar; lucene-demos-2.3.1.jar org.apache.lucene.demo.SearchFiles d:\lucenetest\index

The first step, we input “data*” as query term, then there are 6 files returned. (Asteroid sign equal to wildcard sign, representing that any word which begins with “data”, will returned in query results, such as “database”, “dataware”, “data-base”, and so on). The second step, we input “data” as query term, and then there are 5 files returned. This means that adding query restrictions will increase the precision. The third step, we input “data AND JAVA” as query term, then there is merely one file returned. The fourth step, we input “data AND NOT JAVA” as query term, then there are 4 files returned. The illustration shows as below (Figure 4.12)

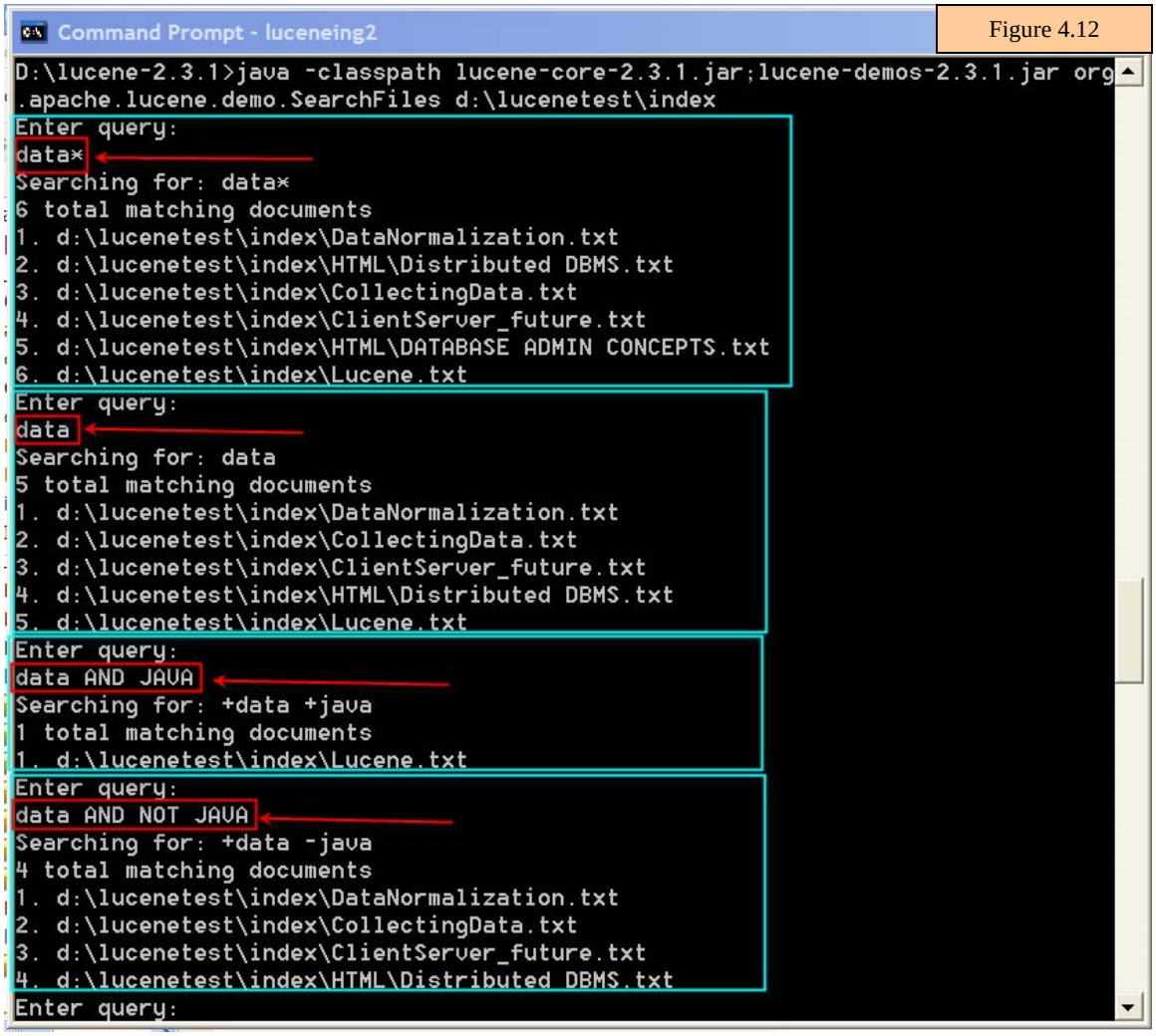



Figure 4.12

### 4.3. Advanced search

Lucene provides some advanced search functions and API.

#### 4.3.1. Multiple indexes

The first one of them is multiple indexes. Multiple indexes in Lucene provide a wonderful way to make a search across distributed indexes and get the suitable results above the interleaving documents (Gospodnetic &Hatcher, 2005). In normal situation, complicated collection sets require many different indexes. Hence, under this situation, multiple indexes can take effect adequately. Moreover, by using Multisearcher class, the search results coming from different indexes are merged together in a special order

(Gospodnetic &Hatcher, 2005). Hence, this technique will become the key base or foundation for multithread search and inter-server search.

The inter-server search takes full advantage of the multiple indexes distributed into different servers and websites. Further more, the computing ability of CPU on different computers in different domain, are in full use. The cooperation among the computers, which is responsible for distributed index and search task, make the search to whole world web site possible and effective. Actually, these kinds of distributed indexes technique are the prototype of Google running. The Figure 4.13 demonstrates the multiple index operation mode of Lucene. The illustration Figure 4.14 demonstrates the surmise about super multiple-index operation mode of Google.

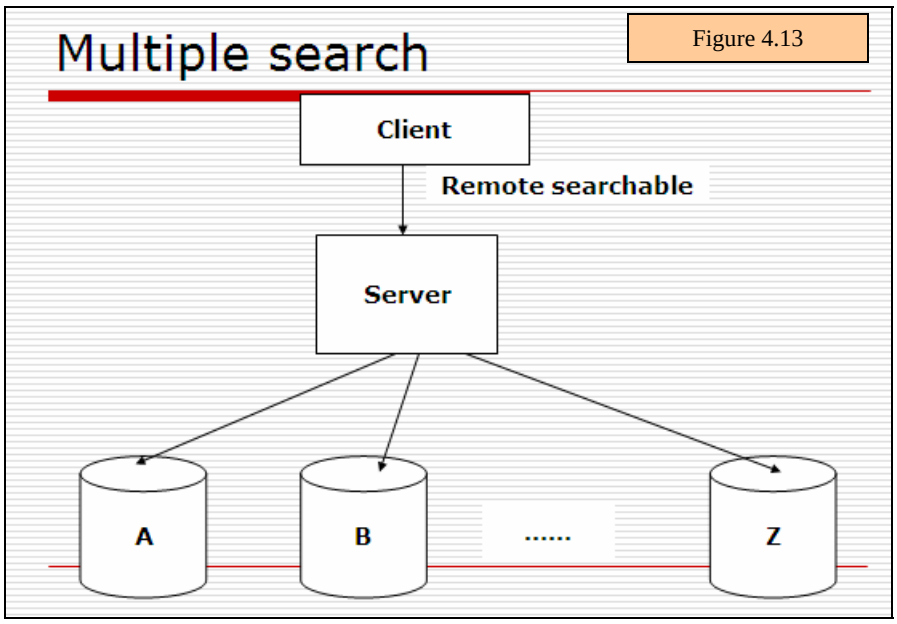



Figure 4.13

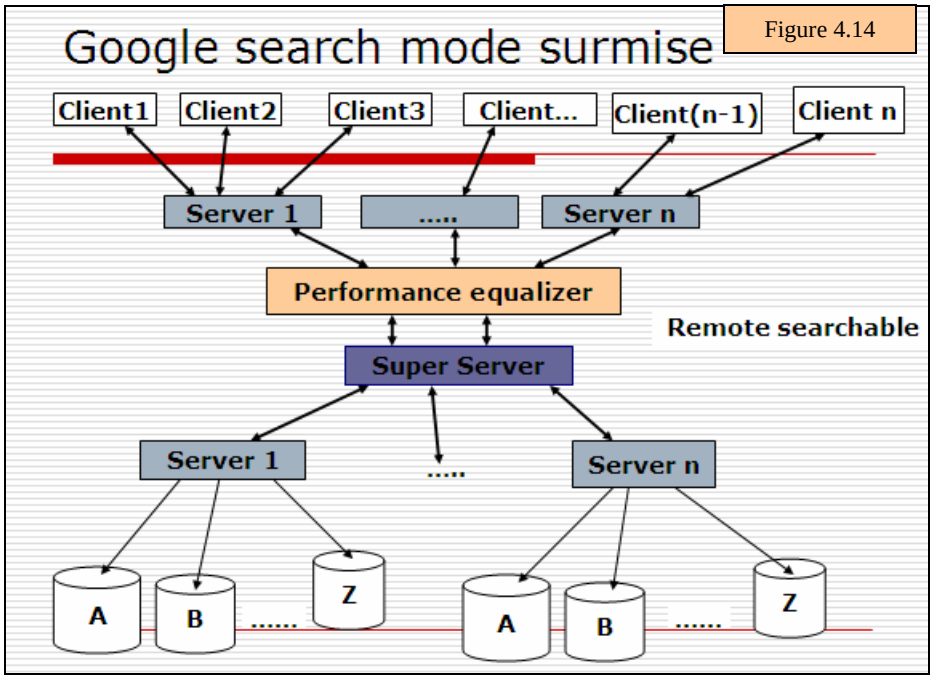
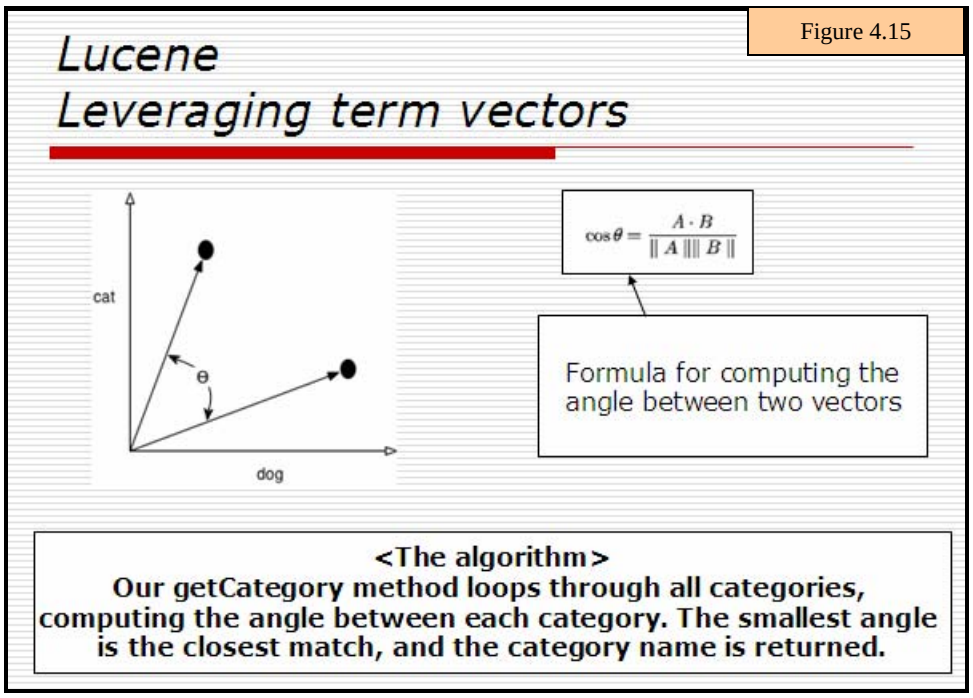



Figure 4.14

### 4.3.2. Term vectors leverage

The second function, in advanced search area, is to compare documents closeness through term vectors.

Lucene employs term vectors to identify the similarity between different documents, and to correspond with users' information needs. For example, there are two documents, which all contain the words of "cat" and "dog" in different frequency. How to estimate the similarity between them is a big problem. However, it can compare them through the angle between different vectors. The more neighboring the angle between them, the more similar two documents are (Baeza-Yates & Ribeiro-Neto, 1999). The illustration below (Figure 4.15) shows the concrete meaning of term vectors, and how to compute them.




However, in order to improve efficiency, Lucene employ algorithms to loop through all of the documents in different categories with the operation of comparison, getting a good effect.

**4.3.3. Customization**

The third functions, in advanced search area, is the customization functions,

The most important customization function, Lucene provided, is to authorize the users to sort search results according to their own will. The first, it compute the distance between search results. The second, it demonstrates the search results in coordination system. The third, it allow users to tailor their own rules to sort results.

Lucene also provide hits collector functions. The first, it is beneficial to develop a wonderful application in analyzing the search hit rates. The second, it provides a good platform to analyze the user's information habit. The third, it makes a preparation for providing customized search ability. Although, this step requires some high-level

application to integrate APIs effectively, the future of it is unbelievable and highly potential.

**4.3.4. Scoring function**

The fourth functions, in advanced search area, is Lucene scoring function. The Score in Lucene is used as the base of sorting search results. Once a Query has been created and submitted to the Index Searcher functions, the scoring process begins to take effect (Apache Lucene researcher, 2005). As for the infrastructure, in the case of Boolean query, scoring functions will and must be implemented by Boolean weight functions. The internal Scorer usually loops over the sub scorers and sums the scores provided by each scorer (Apache Lucene researcher, 2005).

The concrete implementation and formula is as below.

$$\text{score}(q,d) = \text{coord}(q,d) \cdot \text{queryNorm}(q) \cdot \sum_{t \text{ in } q} \left( \text{tf}(t \text{ in } d) \cdot \text{idf}(t)^2 \cdot \text{t.getBoost}() \cdot \text{norm}(t,d) \right)$$

The score of query q for document d has a close relationship with cosine distance computed between documents and query vectors. “A document, whose vector is closer to the query vectors, is marked or scored higher (Apache Lucene researcher, 2005).” There are six variant, requiring to give a detailed introduction.

The first is tf (t in d) which means term frequency, or number of times term t happened in current document. The formula is:

$$\text{tf}(t \text{ in } d) = \sqrt{frequency}$$

The second is idf (t) which means inverse document frequency. It stands for that rarer terms in documents will give higher contribution to total score. The formula is:

$$\text{idf}\ (t) = 1 + \log\left(\frac{\text{number of Docs}}{\text{docFrequency} + 1}\right)$$

The third is coord (q, d) which means “a score factor based on how many of the query terms are found in the specified documents. Typically, a document that contains more of the query's terms will receive a higher score than another document with fewer query terms” (Apache Lucene researcher, 2005).

The fourth is queryNorm (q). It is a normalizing factor employed to make scores from different queries comparable, but not influence document ranking. The formula is:

$$\text{queryNorm}(q) = \frac{1}{\sqrt{\text{sumOfSquaredWeights}}}$$

The sum of squared weights is computed by the query weight object. Boost factors, which are like a tuner, are built into formula to have the ability to affect a query or fields influence on score.

$$\text{sumOfSquaredWeights} = \text{q_getboost}()^2 \times \sum_{t\ \text{in}\ q} (\text{idf}(t) \times \text{t_getboost}())^2$$

The fifth is t_getBoost () which means a search-time running boost of term *t* in the query q, determined by requirements and set by applications.

The sixth is Norm (t, d) which means to encapsulate or put into some boost factors and length factors. Document boost is like a tuner, determined by searchers in advance to adding documents into index. Field boost is determined by searchers before adding fields into a document. Lengthnorm (field) is computed in accordance with the number of tokens on this field of document. When a document is added to index, all the factors are computed and multiplied (Apache Lucene researcher, 2005). The formula is:

$$Norm(t,d) = \text{DocumentBoost} \times lengthNorm(\ filed) \times \prod_{\text{field f in d named as t}} \text{FieldBoost}$$

### 4.4. Advanced function demonstration on Lucene

Some advanced examples in Lucene require Apache-Ant platform to demonstrate. We have installed the apache before. Hence, it is convenient to get results by running Lucene API on Apache-Ant.

#### 4.4.1. Analyzer demonstration

The Lucene analyzer encompasses four sub analyzers. At first, we input the sentence “The information retrieval has a brilliant future” as the query sentence. The first, WhitespaceAnalyzer is responsible to erase the space in query sentence. So, we get the results which are “[The] [information] [retrieval] [has] [a] [brilliant] [future]”, which has no space. The second, SimpleAnalyzer is responsible to turn title case into lower case. So, we get the results which are “[the] [information] [retrieval] [has] [a] [brilliant] [future]”, which are all lower case. The third, StopAnalyzer is responsible to remove the stop words. So, we get the results which are “[information] [retrieval] [has] [brilliant] [future]”. We find [the] and [a] are successfully removed. The illustration shows as below (Figure 4.16).

Figure 4.16

```
D:\LuceneInAction>ant AnalyzerDemo
Buildfile: build.xml

check-environment:

compile:

build-test-index:

build-perf-index:

prepare:

AnalyzerDemo:
     [echo]
     [echo]       Demonstrates analysis of sample text.
     [echo]
     [echo]       Refer to the "Analysis" chapter for much more on this
     [echo]       extremely crucial topic.
     [echo]
    [input] Press return to continue...

    [input] String to analyze: [This string will be analyzed.] [This string will
 be analyzed.]
The information retrieval has a brilliant future.
     [echo] Running lia.analysis.AnalyzerDemo...

     [java] Analyzing "The information retrieval has a brilliant future."
     [java]   WhitespaceAnalyzer:
     [java]     [The] [information] [retrieval] [has] [a] [brilliant] [future.]

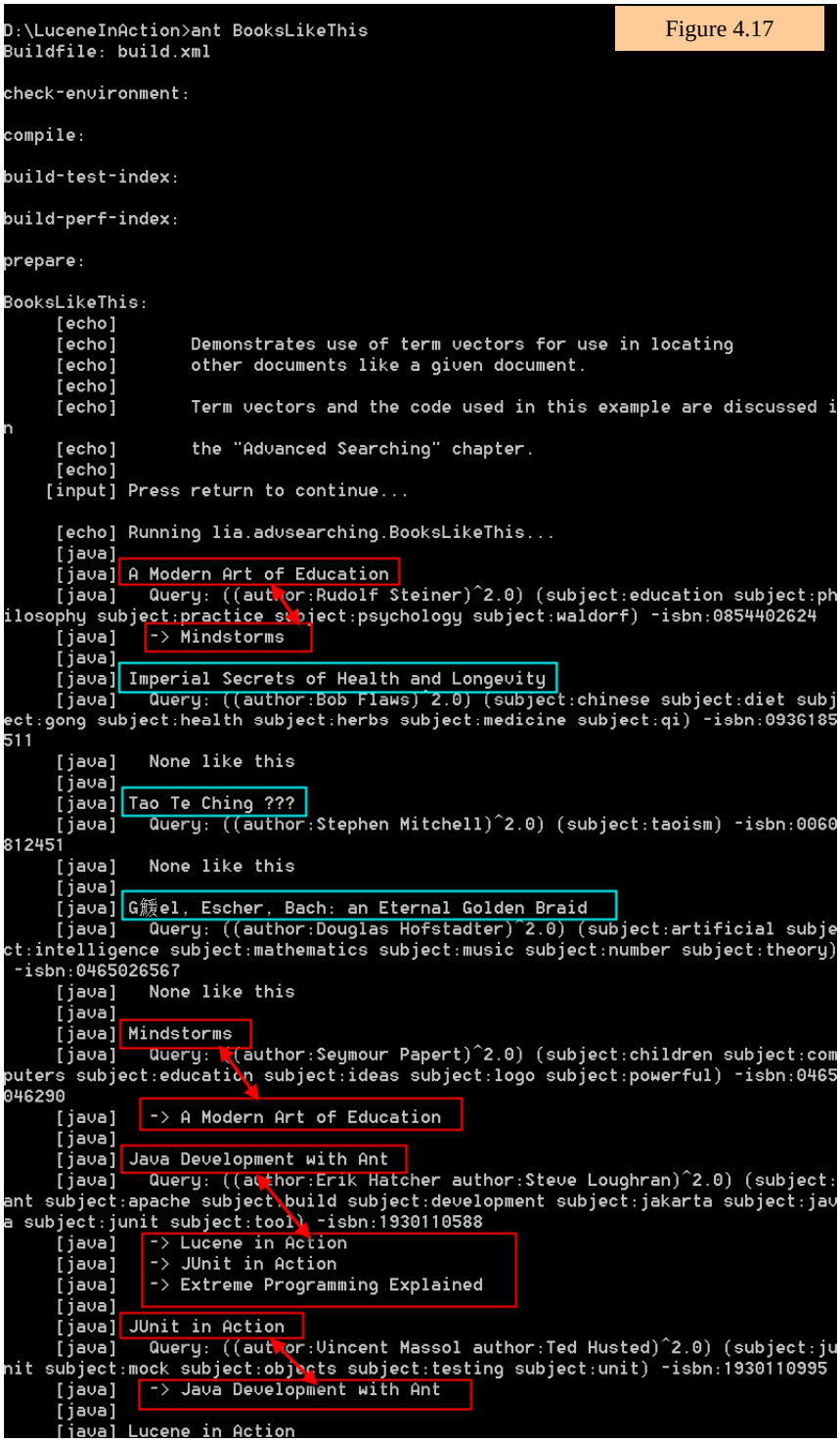
     [java]
     [java]   SimpleAnalyzer:
     [java]     [the] [information] [retrieval] [has] [a] [brilliant] [future]
     [java]
     [java]   StopAnalyzer:
     [java]     [information] [retrieval] [has] [brilliant] [future]
     [java]
     [java]   StandardAnalyzer:
     [java]     [information] [retrieval] [has] [brilliant] [future]
     [java]

BUILD SUCCESSFUL
Total time: 22 seconds
D:\LuceneInAction>
```

### 4.4.2. Vector leverage demonstration

BookLikeThis class in Lucene demonstrates the term vector use (Figure 4.17). The double-arrow depicts which books are related each other, and term vector computation loop through all books. In the illustration below, the books connected by red double arrows are most similar each other. The books marked with blue pane, are none-relevant books.

Figure 4.17




### 4.4.3. Search results Sorting demonstration

Lucene sorting functions, demonstrate how to sort the search results according the computed score (Figure 4.18). The algorithms of score computation have talked in above section.

Figure 4.18

```
D:\LuceneInAction>ant SortingExample
Buildfile: build.xml

check-environment:

compile:

build-test-index:

build-perf-index:

prepare:

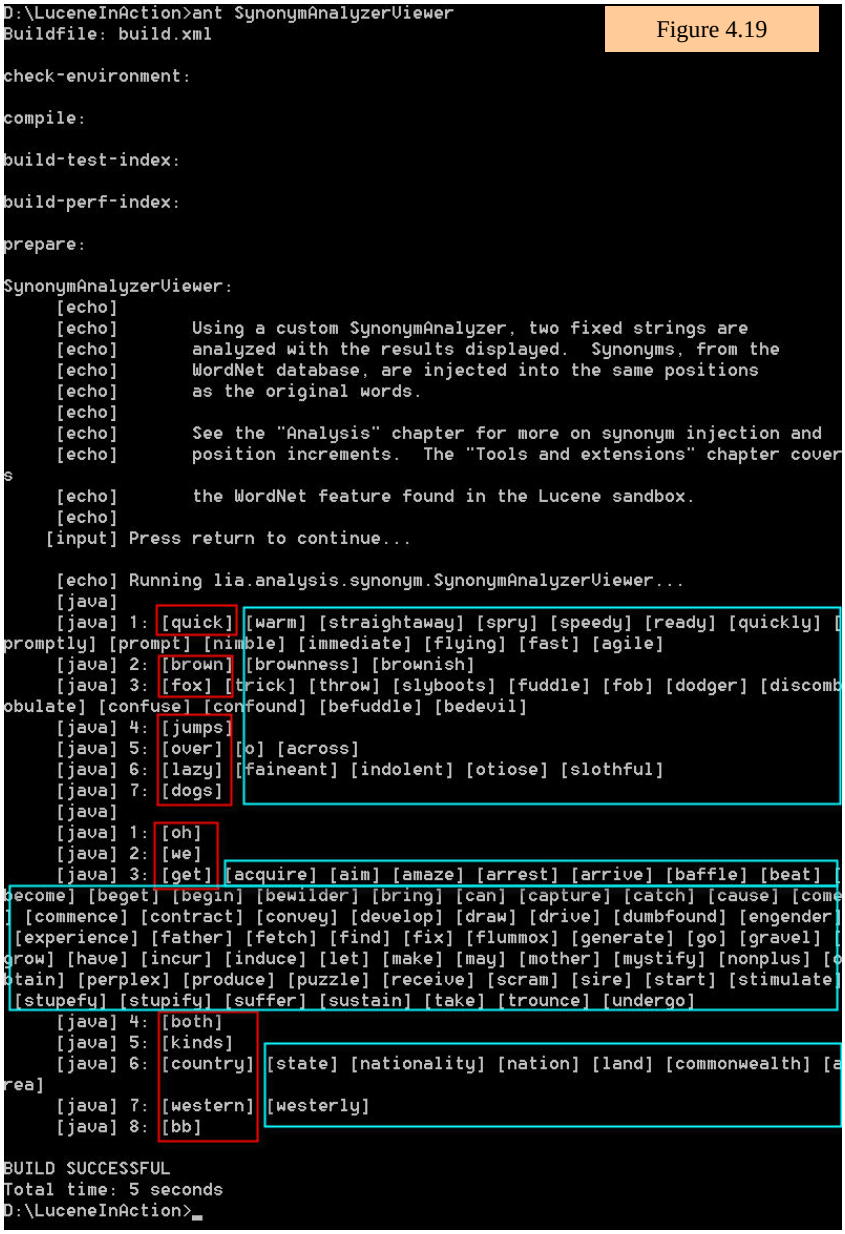
SortingExample:
     [echo]
     [echo]       Lucene's new sorting capabilities are demonstrated by sorting
     [echo]       the same search results in various ways.
     [echo]
     [echo]       Sorting is covered in the "Advanced searching" chapter.
     [echo]
    [input] Press return to continue...

     [echo] Running lia.advsearching.SortingExample...
     [java]
     [java] Results for: pubmonth:[190001 TO 201012] sorted by <score>,<doc>
     [java] Title                          pubmonth   id      score
     [java] A Modern Art of Education      198106     0       0.086743
     [java]    /education/pedagogy
     [java] Imperial Secrets of Health... 199401     1       0.086743
     [java]    /health/alternative/chinese
     [java] Tao Te Ching ???               198810     2       0.086743
     [java]    /philosophy/eastern
     [java] G鐨el, Escher, Bach: an Et... 197903     3       0.086743
     [java]    /technology/computers/ai
     [java] Mindstorms                     198001     4       0.086743
     [java]    /technology/computers/programming/education
     [java] Java Development with Ant      200208     5       0.086743
     [java]    /technology/computers/programming
     [java] JUnit in Action                200310     6       0.086743
     [java]    /technology/computers/programming
     [java] Lucene in Action               200406     7       0.086743
     [java]    /technology/computers/programming
     [java] Tapestry in Action             200403     9       0.086743
     [java]    /technology/computers/programming
     [java] Extreme Programming Explained 199910     8       0.062685
     [java]    /technology/computers/programming/methodology
     [java] The Pragmatic Programmer       199910     10      0.062685
     [java]    /technology/computers/programming
     [java]
```

### 4.4.4. Synonym analyzer demonstration

Lucene synonym analyzer provides the synonym analysis functions. In the illustration below (Figure 4.19), the parts marked with red pane are terms from original sentence. The parts marked with blue pane are synonym terms which are extracted from WorldNet database. The analyzer extracts synonym from WordNet database, then inject them into Lucene index files, on appropriate position connecting with original terms.

Figure 4.19




## 5. Sandbox in Lucene

Sandbox actually is an online environment in which code or content changes can be tested and run without affecting the original system (Wikipedia staff, 2005). The project or area in sandbox of Lucene, involves many aspects, such as using a GUI to interact with index, some flexible analyzer and tokenizers, Java development with Ant, JavaScript browsing functions, synonym handing from WordNet, Highlighting query terms in search

results, chaining filters, and storing index in Berkeley DB. Facing so many sandbox projects, it is impossible to introduce them one by one. Hence, only some important project requires entering into vision in this paper.

Here, one of sandbox projects, named Luke, will be described in detail for its peculiar status in making index structure visible or viewable. The downloading address of Luke is http://www.getopt.org/luke/. The newest version of Luke is 0.8.1. However, because Luke is the project within the sandbox, it must run under the Java platform, meaning a JDK required.

The first step, the index directory must be confirmed. Here, we use D:\lucene-2.3.1\index as the index directory (Figure 5.20).

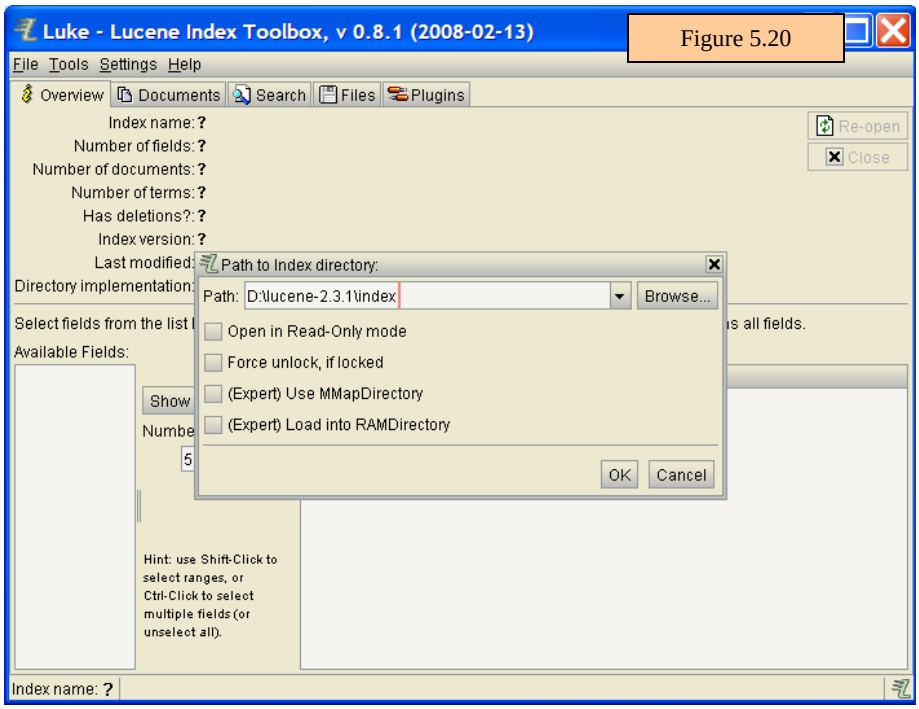



Figure 5.20

The second step, we can see the number of fields, number of documents, and number of terms, available fields, fields name, field content, and top terms in the Luke's interface (Figure 5.21).

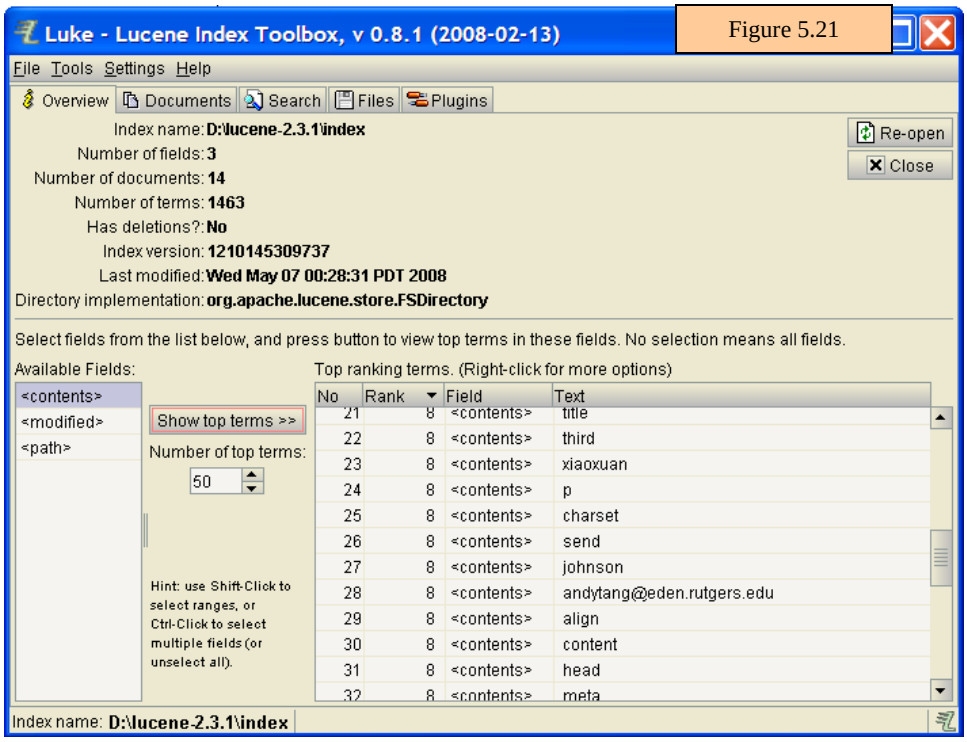


Figure 5.21

The third step, we can browse the modified date, path, and name of every documents (index). Whether the documents are indexed, tokenized, are marked by flags in a clear way. Furthermore, we can search any terms in the index documents, then finding which documents have this term, and the term frequency inside it. The illustration shows as below (Figure 5.22).

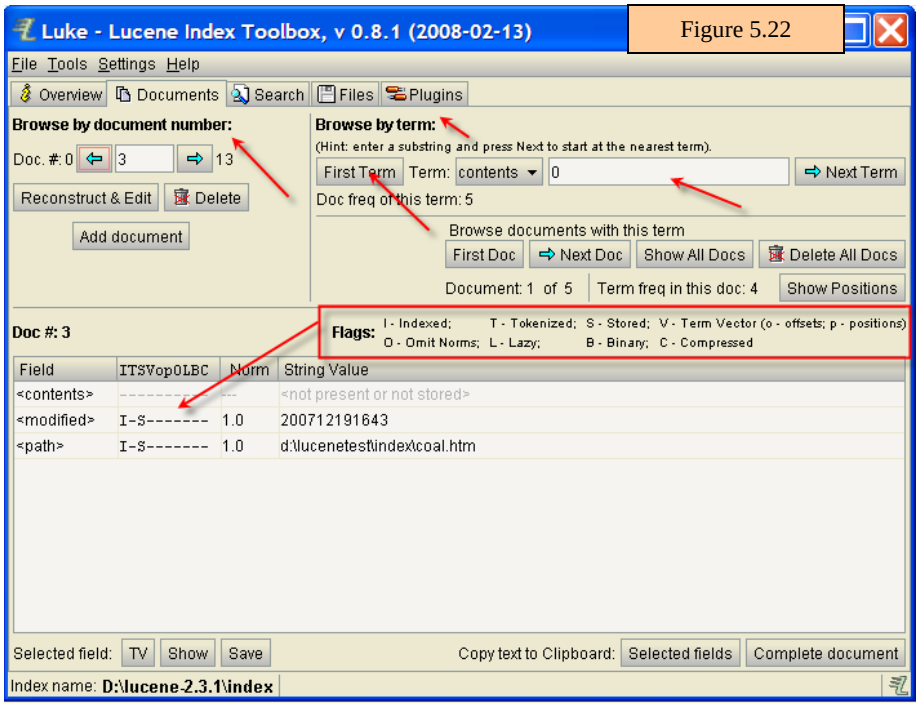


Figure 5.22

It also provides functions to browse the physical characteristics of index files, such as index size, whether in use. The illustration below (Figure 5.23) demonstrates this symptom.

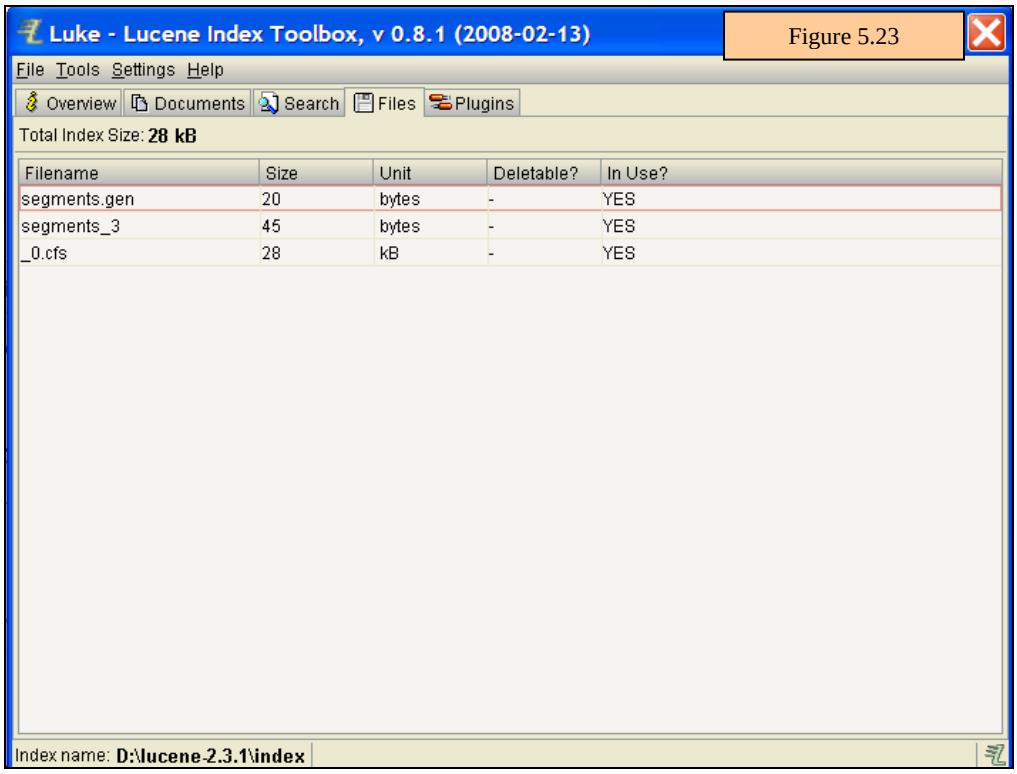


Figure 5.23

In total, Luke is a strong index-viewing tool which definitely provides a friendly interface to pursue index checking.

**6. Different platforms of Lucene**

Because of inter platform characteristics of JAVA, Lucene can run and operate in most hardware platforms, such as UNIX, Linux, Sun Solaris, Windows, and Mac Os. However, because of the complicity of internet and servers environment, there still are some other versions of Lucene developed under different programming languages. They are CLucene on C++ language, dotLucene on C# language, PLucene on perl language, Luby on Python language, Zend Framework on PHP language. There Lucenes on different platforms have different advantage. Among them, the CLucene is the fastest version which is insurmountable. However, the recompiling inconvenience on different hardware platform is a big puzzle for CLucene. Until now, Lucene-java still is the flag product in Lucene series.

**7. Conclusion**

Lucene is a great, excellent, magnificent product on the aspect of information retrieval, whatever on academic target, or on practical usage. The strongpoint of Lucene not only comes from its design schema, but also comes from its open-resource characteristics. In future Lucene will have a trend to penetrate from bottom support layer to top application layer. Most of Lucene functions, such as indexing, searching, query parsing, result representation, algorithms, and multiplatform computation, will grow much stronger and perfect, under the open resources stream.

Some advance IR concepts, which is hardly realized under the limit of computing ability, will be come true under the multiple-thread and multiple-platform support from

Lucene. The easy usability and flexible components of Lucene-java will make it become the strong weapon in solving the problem of information retrieval. The future of Lucene-java is brilliant.

**Reference**

Baeza-Yates, R. & Ribeiro-Neto, B. (1999) *Modern information retrieval*. New York: ACM Press.

van Rijsbergen, C.J. (1979) *Information retrieval,* 2nd ed. London: Butterworths.

Hatcher, E, & Gospodentic, O. (2005) *Lucene in action*. Greenwich: Manning Publication.

Wikipedia staff (2007) http://en.wikipedia.org/wiki/Sandbox, Retrieved 09 May 2008, from http://en.wikipedia.org/wiki/Sandbox

Apache Lucene researcher (2008) Lucene-Apache-scoring, Retrieved 09 May 2008, from http://lucene.apache.org/java/docs/scoring.html

Apache POI researcher. (2008) Apache POI. Retrieved 08 May 2008, from http://poi.apache.org/

Apache Tomcat researcher. (2008) Apache tomcat. Retrieved 08 May 2008, from http://en.wikipedia.org/wiki/Apache_Tomcat

Apache Ant researcher. (2008) Apache Ant. Retrieved 08 May 2008, from http://en.wikipedia.org/wiki/Apache_Ant